
Introduction to Test Design Techniques

Equivalence Class Partitioning

Boundary Value Analysis

Test design techniques

Purpose of test design techniques is to identify test conditions and test cases.

Software testing is complicated, we need to have robust design techniques to enable effective, yet economic/practical testing.

Why use techniques?

Exhaustive testing of all program paths is impractical

Exhaustive testing of all inputs is also impractical

Even if we could, most tests would represent duplicates (that may bring no additional value)

So, we need to select tests which are

- effective at finding faults
- efficient.

Category of Test Design Technique

Test Technique Category	Common features
Specification based Black-Box	Deriving test cases directly from a specification or model of a system/proposed system. Models, either formal or informal, are used for the specification of the problem to be solved, the software or its components. Test cases can then be derived systematically.
Structure based White-box	Deriving test cases directly from the code written (or design) to implement a system. Information about how the software is constructed is used to derive test cases. The extent of coverage of the software can be measured for existing test cases, and further test cases can be derived systematically to increase coverage.
Experience based	Deriving test cases from the tester's experience of similar systems and general experiences of testing. Knowledge of testers, developers, users and other stakeholders about the software, its usage and its environment. They will have knowledge about likely defects and their distribution.

Specification based or black box techniques

- Equivalence partitioning.
 - Boundary value analysis.
 - Decision table testing.
 - State transition testing.
 - Use case testing.
-
- ISO/IEC/IEEE 29119-4:2015 Software and systems engineering — Software testing — Part 4: Test techniques

Equivalence Partitioning

Equivalence partitioning

Can be applied at all test levels.

Is usually recommended as one of the first test techniques to use.

Is based on **dividing a set of test conditions into groups** that can be considered the same, that is, the **system will treat them equivalently**.

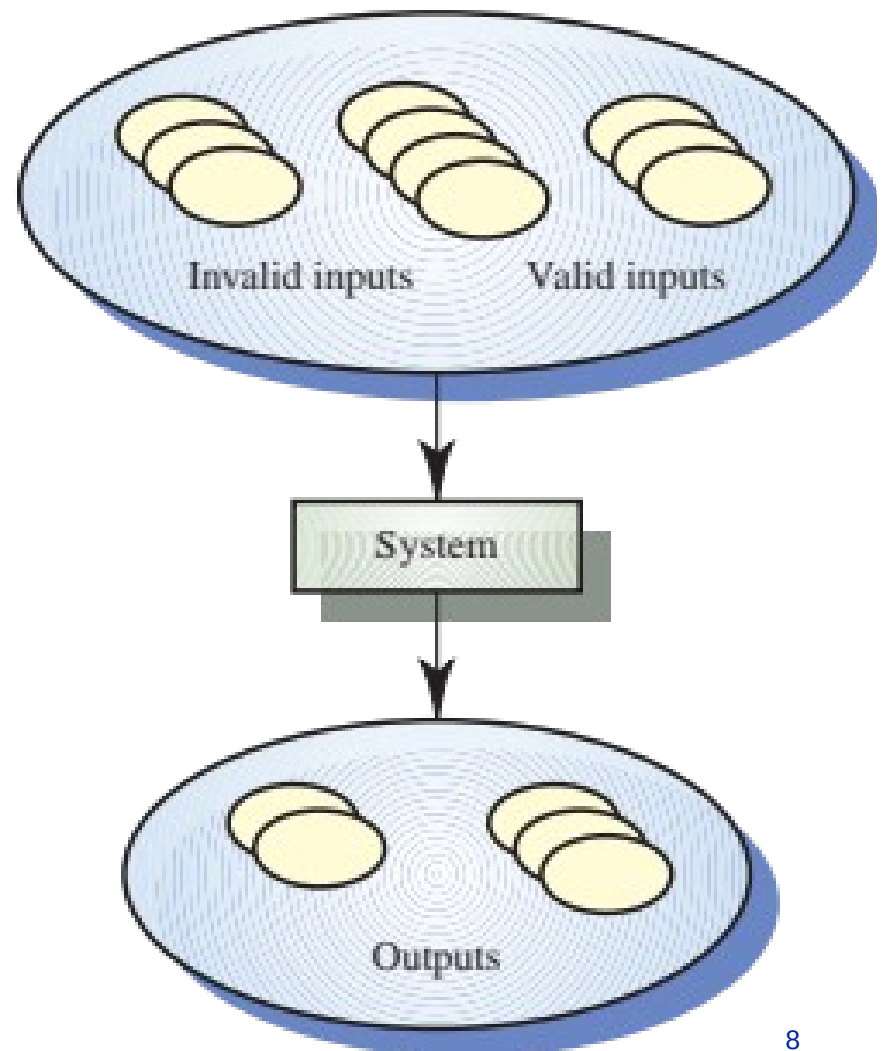
When we have our partitions, we can choose one condition from the partition to test as **we assume that all the conditions in the partition will work the same**.

The result of testing a single value from an equivalence partition is **considered representative of the complete partition**.

This is a very simplified assumption and it is usually advisable to test more than one condition from a particular partition.

Equivalence Partitioning

Equivalence partitioning is the process of methodically reducing the large (or perhaps infinite) set of possible test cases into a **small, but equally effective, set of test cases.**



Valid and Invalid

Equivalence partitions may also be treated in terms of **valid** partitions and **non-valid** partitions.

The particular specification may only mention information that can derive the valid partitions but it is important, as a tester, to be able to identify non-valid partitions as well.

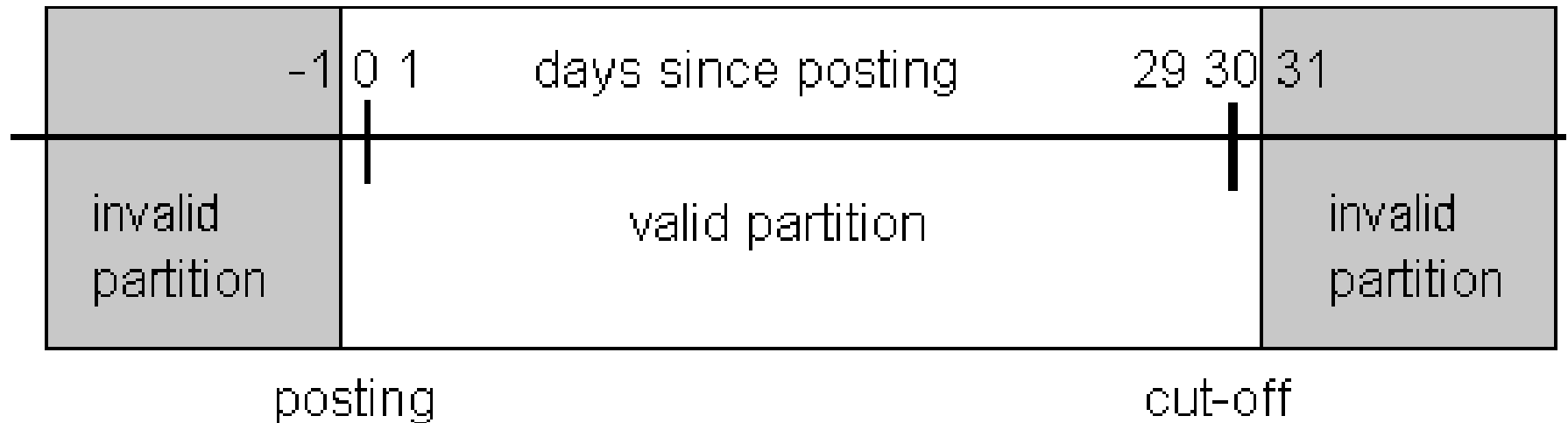
Invalid partition values may still be entered by the user but they are not expected input by the system.

In this case the software should be able to correctly handle values from the invalid partition by displaying the appropriate error message.

Equivalence partitioning example

Requirement:

Contestants have 30 days from date of posting form to return the entry form.



Identifying equivalence partitions

<i>Validity Rule</i>	<i>Input Requirement</i>	<i>Valid Equivalence Class(es)</i>	<i>Invalid Equivalence Classes</i>
Range	Between 0.0 and 99.9	Between 0.0 and 99.9	<0.0 >99.9
Count	Must be 1, 11, 43, 82, 955	1, 11, 43, 82, 955	any number not in the valid list e.g. 2, 37, -56
Set	Must be BUS, CAR or TAXI	BUS CAR TAXI	e.g. XXX, TRAIN, PLANE
Must	Must begin with a letter	Begins with a letter (a-z, A-Z)	Any non alpha character or null e.g. 6, +, >, ~

Identifying the partitions for ranges

Requirement:

“...must be between 23 and 30”

Does between “include” the values 23 and 30?

Implies:

The range 23 to 30 is valid

Less than 23 and greater than 30 are invalid

But does this mean ≤ 22 ?

Or does it mean ≤ 22.99999999 ?

Need to consider the precision of the domain under consideration

e.g. In finance, precision could be very important.

Selecting tests from partitions

When you have identified the partitions, which values should you take?

E.g. in a partition of 1-1000, which value would you choose for a test?

It doesn't matter!

To achieve EP coverage, all that is required is to select ANY value in the range

If we believe the software treats every value in a partition in the same way, the test values in that partition are equivalent, according to our definition.

Test Case Strategy

Once the set of equivalence classes has been identified, here is how to derive test cases:

- Assign a unique identifier to each equivalence class.

- Until all **valid** equivalence classes have been covered by at least one test case, write a new test case covering as many of the valid equivalence classes as possible.

- Until all **invalid** equivalence classes have been covered, write a test case that covers one, and only one, of the uncovered invalid equivalence classes.

- For each test case, annotate it with the equivalence class identifiers that it covers.

Valid and Invalid Classes: When in doubt...

If there is any reason to believe that elements in an equivalence class are not handled in an identical manner by the implementation software, split the equivalence class into further classes.

Equivalence Class Partitioning

Consider creating an equivalence partition that handles the *default*, *empty*, *blank*, *null*, *zero*, or *none* conditions.

Default: no value supplied, and some value is assumed to be used instead.

Empty: value exists, but has no contents.
e.g. Empty string ""

Blank: value exists, and has content.
e.g. String containing a space character " "

Null: value does not exist or is not allocated.
E.g. object that has not been created.

Zero: numeric value

None: when selecting from a list, make no selection.

Equivalence Classes Partitioning - Problems

Specification doesn't always define expected output for invalid test-cases.

Strongly typed languages eliminate the need for the consideration of some invalid inputs.

Brute-force of defining a test case for every combination of the inputs' ECs.

Provides good coverage, but...

...impractical when number of inputs and associated classes is large

Boundary Value Analysis

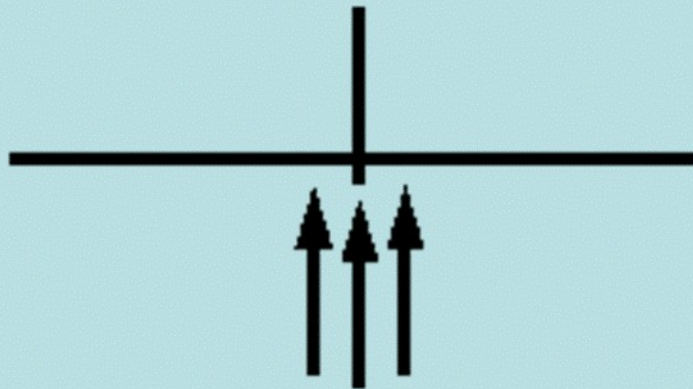
Boundary value analysis

Experience shows **more faults at boundaries of equivalence partitions where partitions are continuous**

So test cases **just above, just below and on the boundary** tend to find faults

Don't forget test cases for output partitions:
just above, on, just below boundaries.

BS5925-2 boundary value analysis



1. Identify the boundary value
 2. Select test values:
 - JUST ABOVE AND
 - JUST BELOW
- ⇒ THREE test values per boundary

Choosing the boundary can be subjective

Can be subjective e.g.

$X < 100$ (choose boundary = 100?)

$X \leq 99$ (choose boundary = 99?)

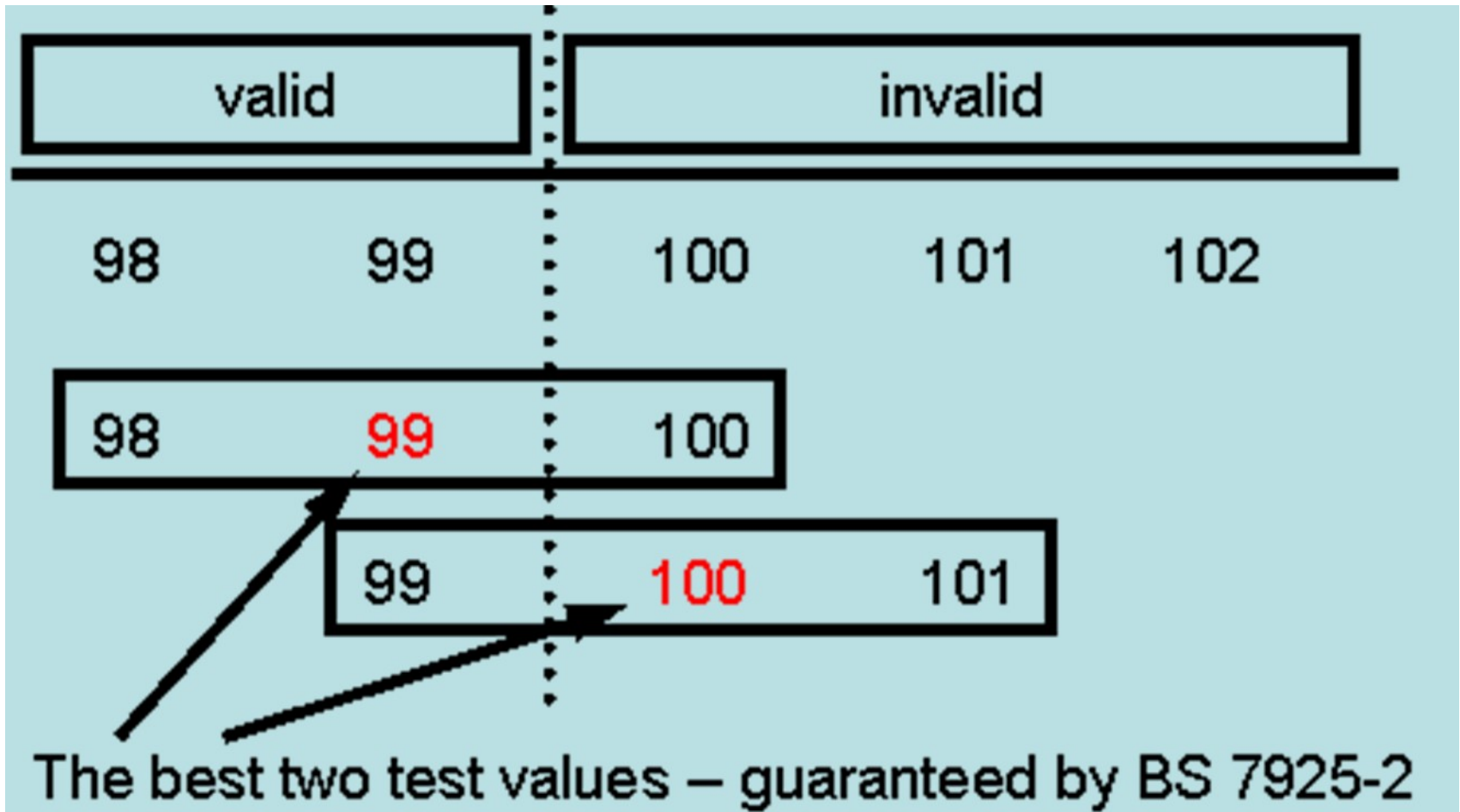
The same requirement, but you could have different test values depending on the boundary:

99, 100, 101 or 98, 99, 100

Which is best?

Because testers can be “one out”, the standard suggests three boundary values (and not two) to guarantee you get the BEST two test cases.

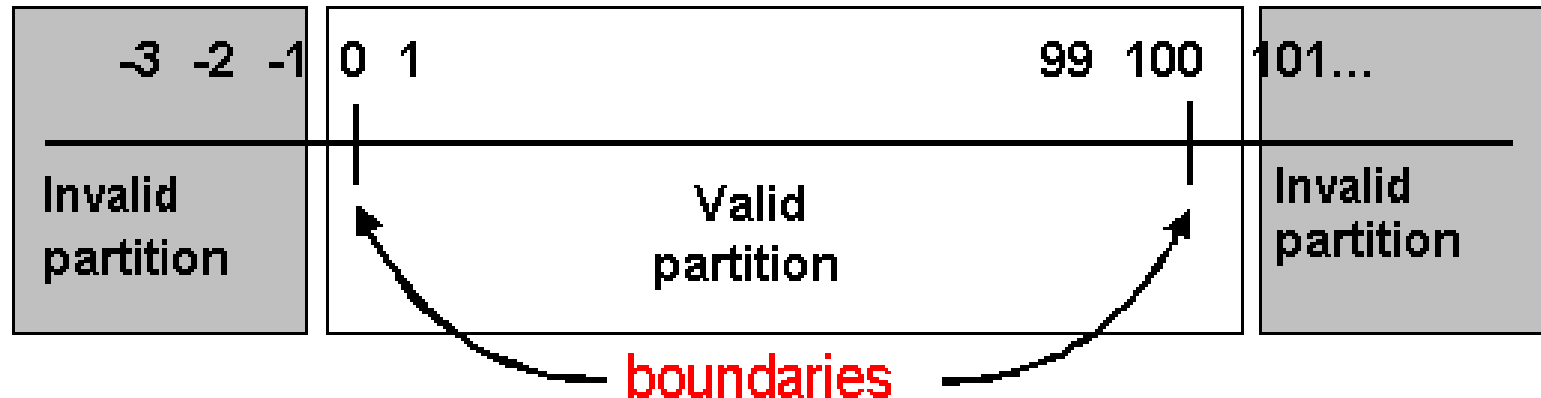
Choosing the boundary can be subjective 2



Boundary value analysis example

Requirement:

A value is valid in the range 0 to 100



Equivalence partitioning test cases: -5, 56, 101

Boundary value analysis: -1, 0, 1, 99, 100, 101

Boundary values subsumes equivalence partitions for ranges

BV subsumes EP where the partitions are ranges of values

If you have covered all boundary values you automatically get EP coverage

You might have chosen different EP values, but since all values in a partition ARE equivalent any test value – even extreme ones - will do it.

Questions

